

## Спецификаторы доступа наследования

### • public

БАЗОВЫЙ

public

protected

private

⇒

Производный

public

protected

X

### • protected

БАЗОВЫЙ

public

protected

private

⇒

Производный

protected

protected

X

### • private

БАЗОВЫЙ

public

protected

private

⇒

Производный

private

private

X



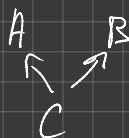
СПАТЬ НЕМНОГО ХОЧЕТСЯ...

## Граф наследования

```
class A {}
```

```
class B {}
```

```
class C: public A, public B {}
```



Будем писать кратко:

```
A;
```

```
B: A;
```

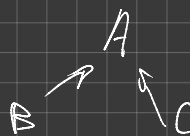
```
C: B;
```



```
A;
```

```
B: A;
```

```
C: A;
```



```
A;
```

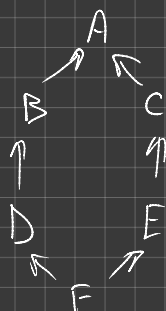
```
B: A;
```

```
C: A;
```

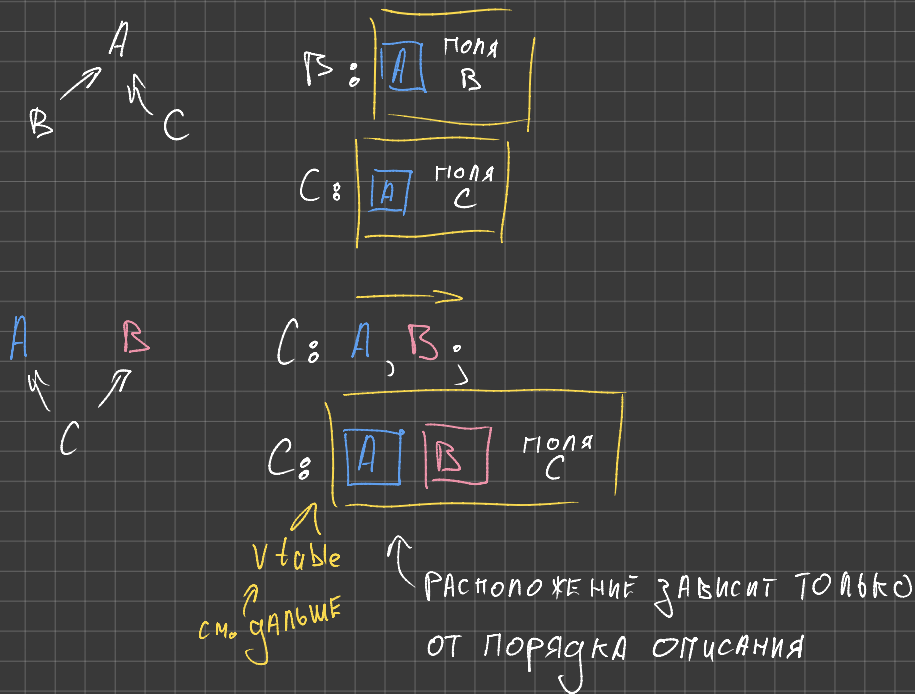
```
D: B;
```

```
E: C;
```

```
F: E, D;
```

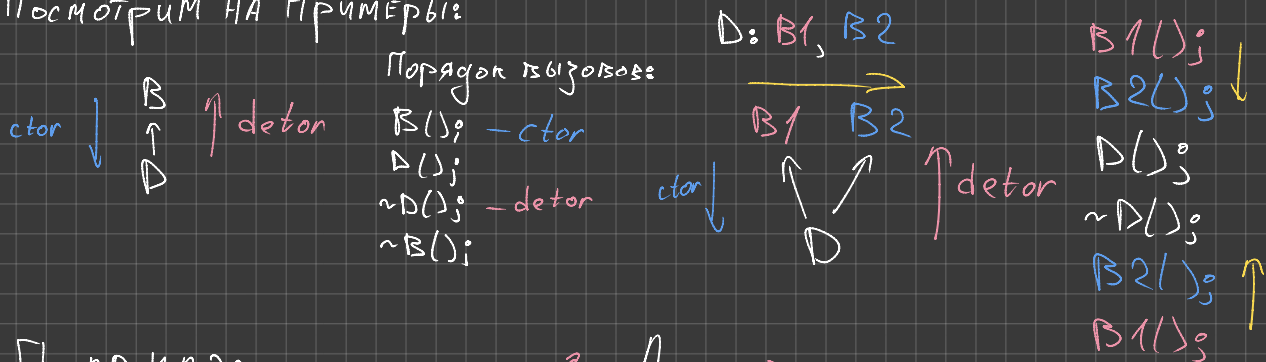


## Укладка полей наследуемого класса в памяти



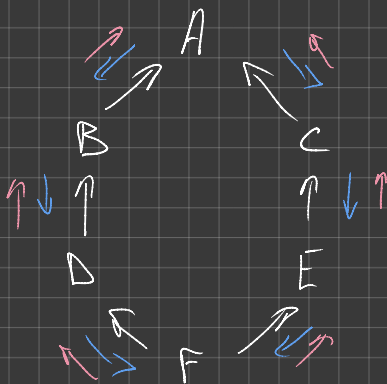
## Конструкторы (ctor) и деструкторы (dctor)

Посмотрим на примеры:



## Правило:

ctor ↓ ↑ ↓  
dctor ↑ ↑ ↓



## Почему так?

- Базовые классы ничего не знают о своих производных;
- Конструкторы производных классов могут использовать для инициализации настройки суперклассов;
- Разрушение базового класса подразумевает разрушение производного;
- Имеет смысл разрушить производный класс до того, как будет уничтожен весь объект

## Передача параметров конструкторам базовых классов

A ctor(x)  
↑  
B ctor(x,y)

```
ctor-производного(params):  
    ctor-БАЗОВОГО-1(params 1),  
    ctor-БАЗОВОГО-2(params 2),  
    ...  
    ctor-БАЗОВОГО-N(params N),  
    поле 1  
    ...  
    поле M } НАШИ поля  
    {  
        Тело_ctor  
    }
```

### Мораль



Конструктор дочернего класса должен принимать параметры, чтобы передать их «транзитом» вверх суперклассам, ДАЖЕ ЕСЛИ У НЕГО НЕ ЗАДУМЫВАЛИСЬ СВОИ ПАРАМЕТРЫ



После двоеточия можно звать конструкторы суперклассов и заполнять свои поля



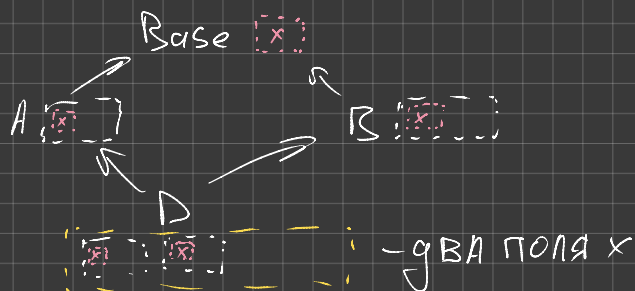
Сначала звать конструкторы суперклассов - в порядке наследования, потом заполнять поля - в порядке объявления

## 4) Полиморфизм

### Виртуальное наследование

У нас проблема: Бриллиантовое наследование  
это также краткий псевдокод

```
Base :  
    public:  
        int x;  
A : public Base;  
B : public Base;  
D : public A, public B;
```



И что?

D d;

d.x = 5; - ОШИБКА: НЕ ЗНАЕМ, КАКОЙ X НУЖЕН

РЕШЕНИЯ

1. Операция разрешения контекста ::

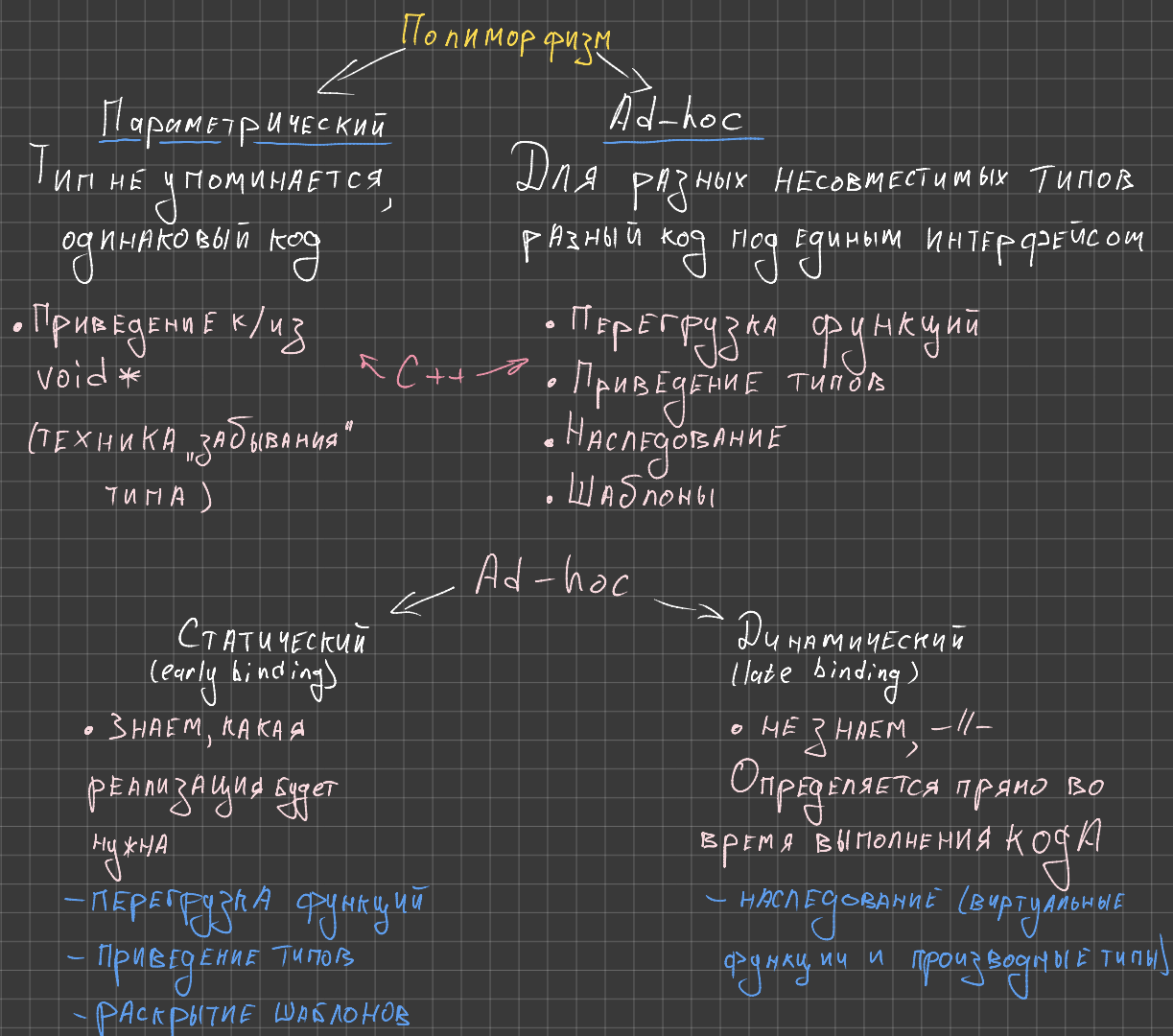
d.A::x = 5;  
d.B::x = -30; | Всё ОК

2. Виртуальное наследование

A: virtual Base; | ГАРАНТИРУЕТСЯ, ЧТО БУДЕТ  
B: virtual Base; | ровно одна копия Base в каждом наследнике

Теперь d.A::x и d.B::x - одно и то же

Полиморфизм — способность функции ОБРАБАТЫВАТЬ данные разных типов

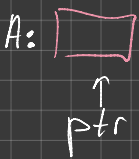


## УКАЗАТЕЛИ НА ПРОИЗВОДНЫЕ ТИПЫ

УКАЗАТЕЛЬ НА БАЗОВЫЙ КЛАСС, МОЖЕТ УКАЗЫВАТЬ НА ЛЮБОГО НАСЛЕДНИКА

❗ ОБРАТНО НЕ РАБОТАЕТ

```
A;  
B: A;  
A* ptr;  
A a;  
B b;  
ptr = &a; | OK  
ptr = &b;
```



ptr (указывает только на наследующую часть)

## КАК ПОЛУЧИТЬ ДОСТУП

$((B*) ptr) \rightarrow func\_in\_B();$

$static\_cast < B* > (ptr) \rightarrow func\_in\_B();$

## Виртуальные функции (virtual)

- МОЖЕТ БЫТЬ ПЕРЕОПРЕДЕЛЕНА В НАСЛЕДНИКАХ
- ПОМЕЧАЕТСЯ virtual, при ПЕРЕОПРЕДЕЛЕНИИ — НЕОБЯЗАТЕЛЬНО  
    ↑  
    В БАЗОВОМ
- КОГ ЗАВИСИТ ОТ ТИПА ОБЪЕКТА

Класс с virtual — ПОЛИМОРФНЫЙ (ТАКЖЕ И НАСЛЕДУЕМЫЕ)

## ПЕРЕОПРЕДЕЛЕНИЕ / ПЕРЕГРУЗКА

- Virtual-функция, которая ПЕРЕОПРЕДЕЛЯЕТСЯ — ПЕРЕОПРЕДЕЛЕННАЯ

- НЕ СЛУЧАЙ ПЕРЕГРУЗКИ

ПЕРЕГРУЗКА → РАЗНЫЕ ПАРАМЕТРЫ

ПЕРЕОПРЕДЕЛЕНИЕ → ОДИНАКОВЫЕ ПАРАМЕТРЫ

РАЗНЫЕ ТИПЫ ОБЪЕКТОВ, ОТ КОТОРЫХ ВЫЗЫВАЕМСЯ

- Virtual func → МЕТОД КЛАССА, НЕ friend (МБ friend для других классов)

## Пример

who() - ВЫВОДИТ СВОЙ КЛАСС

Base:

virtual who(); ← КАКАЯ-ТО РЕАЛИЗАЦИЯ

D1: Base:

who();

D2: D1:

who();

→ ТИПЫ СООТВЕТСТВУЮТ

Base. who();

Out:

D1. who();

Base

D2. who();

D1

D2

changes:

D2: Base:

НЕТ СВОЕЙ РЕАЛИЗАЦИИ who()

Out:

Base

D1

Base

changes:

D2: D1:

-//-

Out:

Base

D1

D1

ПРАВИЛО: идём ВВЕРХ ПО СТРЕЛОЧКАМ ГРАФА НАСЛЕДОВАНИЯ, ПОКА НЕ НАЙДЁМ БЛИЖАЙШУЮ РЕАЛИЗАЦИЮ

## Виртуальные деструкторы

- НЕТ virtual конструкторов
- ЕСТЬ virtual деструкторы
- ГАРАНТИРУЕТСЯ, ЧТО ПОЛИМОРФНЫЙ КЛАСС ПОЛУЧИТ ИМЕННО СВОЙ ДЕКТРУКТОР